

I - Annexe – énoncés des Tps (java avancé)

1. TP sur les threads (java avancé)

1.1. Création et gestion de Threads

Programmer une petite application qui va démarrer et lancer 2 threads en parallèle :

- un premier thread va afficher un par un les nombres pairs de 0 à n (ex : ≤ 25)
- un second thread va afficher un par un les nombres impairs de 1 à n (ex : ≤ 25)

NB : on pourra structurer l'application avec une classe CounterApp (avec méthode main() qui va créer et démarrer les threads) et une classe CounterTaskV1 implémentant Runnable et qui va compter de 2 en 2 en partant de 0 ou de 1 (selon booléen "even/pair" du constructeur).

Exemple de résultat attendu :

0 2 4 6 8 10 1 3 5 7 9 11 13 15 17 19 21 23 25 12 14 16 18 20 22 24

affichage mélangé des nombres pairs et impairs

1.2. Accès concurrents avec ou sans synchronized

Référentiel https://github.com/didier-tp/java_avance ,
projet java/maven multi-modules *tp/tp_java_avance* , sous module *exos_threads* .

La classe **MyNumberConcurrentListV1WithBugs** est sensée démarrer trois threads en parallèle :

- un premier thread va générer (environ toutes les 2ms) 10 nombres aléatoires et les placer dans une liste "numbers"
- un second thread va régulièrement (environ toutes les 2ms) extraire, retirer et afficher les nombres pairs disponibles au sein de la liste "numbers"
- un troisième thread va régulièrement (environ toutes les 2ms) extraire, retirer et afficher les nombres impairs disponibles au sein de la liste "numbers"

Résultats attendus (après correction des bugs) :

```
extracted even numbers : []
extracted odd numbers : []
extracted even numbers : [234, 120, 116, 80]
extracted even numbers : [82, 94, 192, 52, 254, 134]
extracted odd numbers : [173, 133, 241, 97, 179, 121, 229, 41, 33, 9]
extracted even numbers : []
extracted odd numbers : [71, 213, 185, 243, 43, 215, 183]
extracted even numbers : [30, 44, 204]
extracted even numbers : [52, 2, 206, 88, 198]
extracted odd numbers : [73, 93, 199, 57, 99]
extracted odd numbers : []
extracted even numbers : [122, 38, 190, 218, 214, 122, 242, 160]
extracted odd numbers : [217, 119, 161, 233, 19]
...
```

Gérer l'exception ConcurrentModificationException avec le pattern "collect & removeAll after"

loop" et en introduisant `synchronized(...)` { ... } aux endroits appropriés .

1.3. Gestion de la concurrence (avec sémaphore ou autres)

Reprendre le premier exercice (avec CounterApp) .

Créer une version améliorée *MyCounterTaskV2* qui va **alterner les affichages des valeurs impaires et paires** . On pourra par exemple se baser sur des **sémaphores** *readyEvenMutexAsSemaphore* et *readyOddMutexAsSemaphore* et des sous méthodes *waitOurTurn()* et *notifyYourTurn()* .

Exemple de résultat attendu :

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

1.4. Gestion des DeadLocks (dîner des philosophes)

Référentiel https://github.com/didier-tp/java_avance ,
projet java/maven multi-modules *tp/tp_java_avance* , sous module *exos_threads* .

Problème classique "dîner des philosophes" pour montrer un cas de "blocage mutuel" / "deadlock" et une manière de résoudre le problème .



N(ex:5) philosophes sont dans un cycle :

- penser (en un temps fini et potentiellement un peu aléatoire)
- avoir faim et attendre deux fourchettes libres (à gauche puis à droite)
- manger (en un temps fini et potentiellement un peu aléatoire)
- rendre les fourchettes

Code de départ :

DinerPhilosophesApp.main() avec PhilosopherTaskWithDeadLock

A faire en Tp :

- dupliquer PhilosopherTaskWithDeadLock en PhilosopherTaskWithoutDeadLock
- utiliser PhilosopherTaskWithoutDeadLock dans DinerPhilosophesApp.main()
- ajouter dans le code de PhilosopherTaskWithoutDeadLock un nouveau sémaphore (ex : roomSemaphore) de taille initiale N-1 et ajouter au sein de la méthode run :
 - * une acquisition d'un jeton de "roomSemaphore" pour pouvoir rentrer dans la salle (réfectoire) de manière à pouvoir ensuite tenter de prendre des fourchettes et manger
 - * une restitution d'un jeton dans "roomSemaphore" après "repas pris" et "fourchettes rendues"

Solution :

DinerPhilosophesApp.main() avec PhilosopherTask

1.5. Monitoring , détection et correction de deadlock

- 1) Refaire fonctionner l'application DinerPhilosophesApp avec PhilosopherTaskWithDeadLock
- 2) lancer **jps** et **jstack** (ou autre) pour visualiser les 5 threads (Thread-0 , ... Thread-4) bloqués en attentes mutuelles
- 3) dupliquer PhilosopherTaskWithDeadLock en PhilosopherTaskWithoutDeadLockV2 et utiliser PhilosopherTaskWithoutDeadLockV2 dans DinerPhilosophesApp.main()
- 4) au lieu d'utiliser un sémaphore complémentaire "roomSemaphore" , tenter de résoudre le problème du "deadlock" via tryLock() soit via **tryAcquire()** avec des sémaphores.

1.6. Etude de cas "marché boursier"

Référentiel https://github.com/didier-tp/java_avance ,
projet java/maven multi-modules tp/tp_java_avance , sous module market_core .

Point d'entrée du code : tp.market.app.BasicMarketAppV1

...

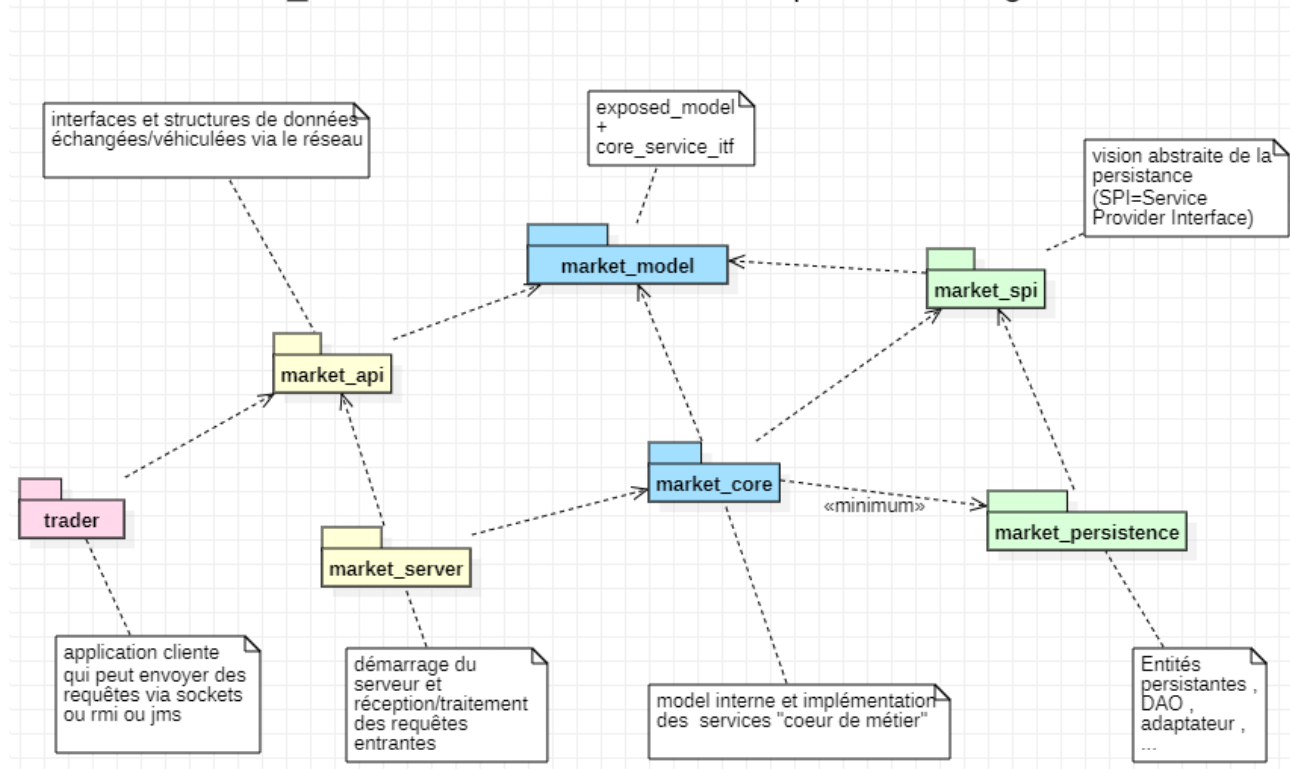
2. TP sur les communications réseau (java avancé)

Référentiel https://github.com/didier-tp/java_avance ,
projet java/maven multi-modules **tp/tp_java_avance**

Structure du projet (à compléter) :

market_core et market_model	Coeur de métier (services et données internes en partie exposés)
market_api	Sous projet avec structures de données communes entre les cotés "serveur" et "client"
market_server	Serveur (place de marché boursier)
trader	Application cliente vis à vis du server "market"

Stock_market en mode "architecture pseudo-hexagonale"



Point d'entrée du code (coté marker_server):

tp.market.my_server.**MyBasicMarketServerAppV2** (début de TP à compléter)
tp.market.server.BasicMarketServerAppV2 (solution possible du TP)

But commun des Tps (aussi bien en version "socket" , "RMI" ou "JMS") :

En s'appuyant principalement sur le sous projet "market_api" , coder les cotés serveur et client de manière à pouvoir :

- envoyer une requête de type "récupérer liste des actions/stocks" .
- récupérer la réponse du coté application cliente "trader"

- générer des ordres d'opérations boursières au sein de l'application "trader" en s'appuyant sur `tp.market.core.service.OrderServiceImpl(List<Stock> stockList)`
- envoyer du client "trader" vers "market_server" un paquet d'ordre (via une boucle adéquate)
- appliquer les ordres reçus du côté "market_server" en déléguant "sur partie modele/core /".

2.1. Communications via sockets

Coté serveur (projet "market_server") :

- votre code à coder et placer dans `tp.market.my_server.my_task.MyOrderCollectorSocketTask` avec `MyBasicMarketServerAppV2.main()` qui déclenche `m_server.my_task.MyOrderCollectorTaskSocketV2` (démarrage de la partie "serveur_socket") qui délègue lui même à `my_server.my_task.MyOrderCollectorSocketTask` la gestion des requêtes.
- solution possible dans `tp.market.server`

Conseils :

On pourra s'appuyer sur les classes du package `tp.market.message` du sous projet "market_api"

Coté client (projet "trader") :

- votre code à coder et placer dans `tp.market.my_trader.MyTraderSocketClient`
- solution possible dans `tp.market.trader.TraderSocketClient`

2.2. Communications via RPC/RMI

Coté serveur (projet "market_server") :

- votre code à coder et placer dans `tp.market.my_server.my_task.MyOrderCollectorTaskRmiV2` avec `MyBasicMarketServerAppV2.main()` qui déclenche `my_server.my_task. MyOrderCollectorTaskRmiV2` (démarrage de la partie "serveur_rmi")
- solution possible dans `tp.market.server`

Conseils :

- interfaces "remote" dans `tp.market.rmi_api` du projet "market_api"
- implémentations prévues : classes dans `tp.market.my_server.my_rmi` du projet "market"

Coté client (projet "trader") :

- votre code à coder et placer dans `tp.market.my_trader.MyTraderRmiClient`
- solution possible dans `tp.market.trader.TraderRmiClient`

2.3. Communications asynchrones via JMS

Coté serveur (projet "market_server") :

- votre code à coder et placer dans `tp.market.my_server.my_task.MyOrderCollectorTaskJmsV2` avec `MyBasicMarketServerAppV2.main()` qui déclenche `my_server.my_task. MyOrderCollectorTaskJmsV2` (démarrage de la partie "serveur_jms")
- solution possible dans `tp.market.server`

Conseils :

- les valeurs des messages JMS (en mode texte) pourront éventuellement être au format JSON en s'appuyant sur `tp.market.util.MyJsonUtil` du sous projet "market_model"
- les types fonctionnels des messages JMS échangés pourront éventuellement être basés sur `tp.market.jms.MarketMessage` et véhiculés via `message.get_or_setStringProperty(..)`

Coté client (projet "trader") :

- votre code à coder et placer dans `tp.market.my_trader.MyTraderJmsClient`
- solution possible dans `tp.market.trader.TraderJmsClient`

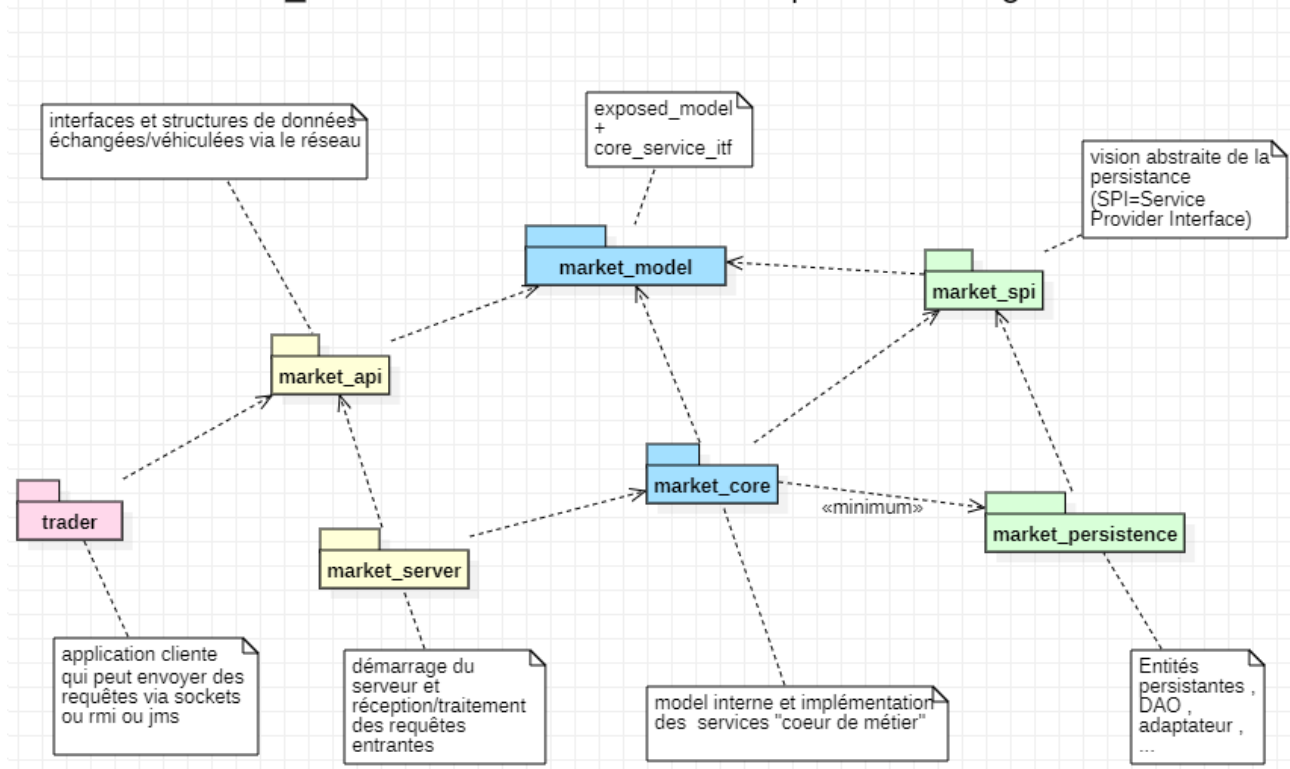
3. TP sur la persistance (JPA/Hibernate) (java avancé)

Référentiel https://github.com/didier-tp/java_avance ,
projet java/maven multi-modules **tp/tp_java_avance**

Structure du projet (à compléter) :

market_spi	Vision abstraite de la persistance vue par le coeur de métier (SPI, ...)
market_core et market_model	Coeur de métier (services et données internes en partie exposés)
market_persistence	Lien avec la base de données (code basé sur jpa/hibernate)

Stock_market en mode "architecture pseudo-hexagonale"



Point d'entrée du code des TPs:

projet "**market_persistence**" (début de TP à compléter)

3.1. Analyse de la configuration java/maven de JPA/Hibernate

* Repérer toutes les parties "jpa/hibernate" dans **pom.xml** du projet "market_persistence" (ainsi que le driver jdbc "mysql-connector-j").

* Visualiser le contenu de src/main/resources/META-INF/persistence.xml et si besoin adaptez ce fichier (mot de passe , url , ...)

* Au sein d'une application java d'entreprise basée sur le framework SpringBoot ou sur le framework JakartaEE , JPA/Hibernate est déjà bien intégré (par exemple via @PersistenceContext et @Transactional).

Dans le projet "market_persistence" (volontairement basé que sur le coeur du langage java) , on n'a pas de transaction automatique ni d'ouverture/fermeture automatique du "EntityManager" et l'on pourra s'appuyer sur la classe tp.market.persistence.jpa.**MyJpaUtil** pour intégrer JPA/hibernate dans notre application java .

* Visualiser (en diagonale) le contenu de src/main/java/tp.market.persistence.generic.**GenericDAO** et **GenericDaoJpa** .

=====

Via le driver jdbc "mysql-connector-j" on pourra se connecter à une base de données **MariaDB** (plus facile à installer que MySQL).

Lors d'une éventuelle rapide installation de MariaDB , on pourra choisir root/root comme username/password .

3.2. Persistance élémentaire

* Compléter tp.market.persistence.my_entity.**MyStockEntity** avec avec @Entity , @Table , @Id , @Column , ...

NB : noms java en CamelCase et nom de tables/colonnes en snake_case à préciser via @Table et @Column .

* ajouter cette ligne (en version "My...") dans src/main/resources/META-INF/persistence.xml (sans enlever l'autre version "solution" sans "My") .

```
<class>tp.market.persistence.my_entity.MyStockEntity</class>
```

* Visualiser le contenu de src/main/java/tp.market.persistence.my_dao.**MyStockDAO** et **MyStockDaoJpa** .

* compléter le code de la classe src/main/test/tp.market.persistence.my_dao.**TestMyStockDAO**

* lancer ce test et corriger les éventuelles erreurs jusqu'à ce que les tests fonctionnent tous correctement.

3.3. @ManyToOne , @OneToMany , transactions et cascades

* Compléter `tp.market.persistence.my_entity.MyStockExchangeEntity` avec avec `@Entity` , `@Table` , `@Id` , `@Column` , ... et `@ManyToOne` , `@JoinColumn`

NB : noms java en CamelCase et nom de tables/colonnes en snake_case à préciser via `@Table` et `@Column` .

* ajouter cette ligne (en version "My...") dans `src/main/resources/META-INF/persistence.xml` (sans enlever l'autre version "solution" sans "My") .

```
<class>tp.market.persistence.my_entity.MyStockExchangeEntity</class>
```

* Compléter le code de `tp.market.persistence.my_entity.MyStockEntity` en y ajout la relation inverse (avec `@OneToMany`)

* Ajouter au sein de `src/main/java/tp.market.persistence.my_dao` l'interface `MyStockExchangeDAO` et la classe `MyStockExchangeDaoJpa` .

* compléter le code de la classe

`src/main/test/tp.market.persistence.my_dao.TestMyStockExchangeDAO`

* lancer ce test et corriger les éventuelles erreurs jusqu'à ce que les tests fonctionnent tous correctement.

* ajouter un déclenchement d'une suppression de la deuxième action (stock) dans la méthode de test.

* ajouter si besoin une **cascade** appropriée près de `@OneToMany` de manière à ce que la suppression de passe bien lors du déclenchement du test .

3.4. Requêtes (JPQL , api Criteria, LAZY, ...)

* Ajouter au sein de `my_dao.MyStockExchangeDAO` et `my_dao.MyStockExchangeDaoJpa` plein de variantes de la méthode de recherche des échanges associés à un `stockId` spécifique :

```
List<MyStockExchangeEntity> findByStockId(String stockId); //JPQL
```

```
List<MyStockExchangeEntity> findByCriteriaStockId(String stockId); //criteria api
```

```
List<MyExchangeEssential> findExchangeEssentialListByStockId(String stockId);  
//with projection/aggregation
```

* tester toutes ces variantes (via changement de lignes en commentaire) au sein de la classe `my_dao.TestMyStockExchangeDAO`

* ajouter dans **MyStockDAO** et **MyStockDaoJpa** et coder une nouvelle méthode telle que

public MyStockEntity findByIdWithExchanges(String stockId)

retournant d'un coup une action et ses échanges rattachés

* ajouter dans **TestMyStockExchangeDAO** une nouvelle variante basée sur un appel de cette nouvelle méthode et tester plein de combinaisons fonctionnelles (EAGER , LAZY et SELECT avec LEFT JOIN FETCH) en privilégiant à la fin celle basée sur LAZY .

3.5. Sécurité et optimisations

* Ajouter au sein de `my_dao.MyStockExchangeDAO` et `my_dao.MyStockExchangeDaoJpa` plein de variantes de la méthode de recherche des échanges associés à un `stockId` spécifique :

```
List<MyStockExchangeEntity> findByStockId(String stockId); //good JPQL with setParameter()
```

```
List<MyStockExchangeEntity> badFindByStockIdWithSqlInjectionProblem(String stockId);
```

```
//wrong JPQL query without .setParameter()
```

Au sein de `TestMyStockExchangeDAO` tester la robustesse des différentes versions via des appels temporaires de ce genre :

```
List<MyStockExchangeEntity> exchanges =
stockExchangeDAO.badFindByStockIdWithSqlInjectionProblem(stockId1+" OR 1=1");
//avec injection SQL !!!
```

* Retirer temporairement le paramétrage `indexes = @Index(name="idx_isin", columnList = "isin", unique = true)` au sein de `@Table` dans la solution `entity.StockEntity`

et lancer un test

* Visualiser la structure de la table "stock" de la base de données via un utilitaire adéquat (ex : "HeidiSql" ou autre).

* ajouter `indexes = @Index(name="idx_isin", columnList = "isin", unique = true)` au sein au sein de `@Table` dans `my_entity.MyStockEntity` et relancer un test.

* Visualiser la nouvelle structure de la table "stock" de la base de données via un utilitaire adéquat (ex : "HeidiSql" ou autre) :

Sans index

De base

Options

Index (1)

Clés étrangères

Ajouter

Supprimer

Effacer

Monter

Descendre

Nom

> PRIMARY KEY

Colonnes :

Ajouter

Supprimer

Monter

# ^	Nom	Type de données
1	current_quote	DOUBLE
2	number	BIGINT
3	isin	VARCHAR
4	name	VARCHAR
5	symbol	VARCHAR

Avec index

De base

Options

Index (2)

Clés étrangères

Ajouter

Supprimer

Effacer

Monter

Descendre

Nom

> PRIMARY KEY

> idx_isin

Colonnes :

Ajouter

Supprimer

# ^	Nom	Type de données
1	current_quote	DOUBLE
2	number	BIGINT
3	isin	VARCHAR
4	name	VARCHAR
5	symbol	VARCHAR

- * Coder la nouvelle méthode stockDAO.**findByIsin(...)** dans la partie my_dao
- * Appeler cette méthode via une petite variante de code au sein du my_dao.**TestMyStockDAO.testCrudStock()**
- * Repérer toutes les parties "**cache**" dans **pom.xml** du projet "market" ...
- * Visualiser le fichier src/main/resources/**ehcache.xml**
- * Visualiser les propriétés hibernate nécessaires au cache dans META-INF/persistence.xml
- * coder une nouvelle classe de test my_dao.MyTestCache avec une méthode de ce genre :

```
@Test
public void testSaveAndFindWithCache(){
    String pk="sxx.PA";
    stockDAO.saveNew(new MyStockEntity("Sxx" , pk , "FR12367890" , 3.03 ));
    String pk2="syy.PA";
    stockDAO.saveNew(new MyStockEntity("Syy" , pk2 , "FR1236780" , 4.03 ));
    long t1 = System.currentTimeMillis();
    MyStockEntity sRelu = stockDAO.findById(pk);
    long t2 = System.currentTimeMillis();
    Cache cache = emf.getCache();//2nd Level cache (shared by all users/threads/sessions/entityManager)
    boolean wasCached = cache.contains(StockEntity.class, pk);
    log.info("findById execution time (with or without cache)="+ (t2-t1));
        //ex: 12ms with cache , 35ms without cache
    log.info("sRelu="+sRelu);
    log.info("wasCached="+wasCached);
    assertEquals("Sxx",sRelu.getName());
    assertTrue(wasCached);
}
```

- * Lancer plusieurs fois ce test sans ou avec l'ajout de

```
@Cacheable
@org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
dans my_entity.MyStockEntity
```

et visualiser d'éventuelles différences .

- * éventuelles autres expérimentations selon l'inspiration du moment ...

4. TP sur coeur langage java (fonctionnalités avancés)

NB : tous ces énoncés de TP sont basés sur le point de départ suivant :

sous projet "exos_java" de tp/tp_java_avance (https://github.com/didier-tp/java_avance)

4.1. TP sur réflexivité

Coder la méthode **generateLineOfCsv()** de la classe `tp.util.TpCsvGenerate` en s'appuyant sur la réflexivité du langage java.

Phase 1 : le fichier .csv (ou bien une de ses lignes) comportera tous les champs existants dans la classe java (ex : `Product` ou `Person`).

Phase 2 : on effectuera le test "est que le champ comporte l'annotation `@CsvIgnore` ?" et si oui ce champ ne sera pas placé dans la sortie (de type `String`) au format csv .

Tester tout cela via `TpCsvGenerate.main()` .

Type de résultat attendu (à la console) pour la phase 1 :

id;firstName;lastName;email;age;taille

1;jean;Bon;jb@gmail.com;40;180

id;label;price;features

1;prod 1;123.0;Bon produit

Fichier `products.csv` (généré par défaut à la racine des projets) pour la phase 2 :

id;label;price

1;printer z1;156.8

2;usb wire;6.8

3;computer x2;976.5

4;webcam;96.83

5;computer y6;896.83

5. TP sur fonctionnalités récentes (java avancé)

NB : tous ces énoncés de TP sont basés sur le point de départ suivant :

sous projet "exos_java" de tp/tp_java_avance (https://github.com/didier-tp/java_avance)

5.1. Eventuelles révisions sur lambda et streams

Si besoin de réviser lambda et stream , coder une partie d'application qui va enchaîner filtrage , tri et transformation sur une liste d'objets structurés de type Produit.

Point de départ du code (à compléter) :

tp.avec_lambda.**AppGestionProduitsAvecStream**
en s'appuyant tp.data.**Product** et tp.util.**ProductUtil**

5.2. Tp sur principales syntaxes récentes (switch , yield , record , textBloc , ...)

Compléter le code de **tp.j15_16_17.SyntheseTestApp** en utilisant au maximum les nouvelles syntaxes des versions 8,9,10,11,12,13,14,15,16,17 de java

Phase 1 :

switch/case avec lambda expression au sein de **tp.j15_16_17.SyntheseTestApp**

Résultat attendu :

LIGNE
RECTANGLE
CERCLE
LIGNE
TYPE DE FIGURE INCONNU

Phase2 :

switch/case avec lambda expression et yield au sein de **tp.j15_16_17.SyntheseTestApp**

Résultat attendu :

LIGNE
resTransformation=Line from Figure(type=line, x1=10, y1=10, x2=100, y2=100, color=red)
RECTANGLE
resTransformation=Rectangle from Figure(type=rectangle, x1=5, y1=5, x2=120, y2=120, color=blue)
CERCLE
resTransformation=Circle from Figure(type=circle, x1=40, y1=40, x2=80, y2=80, color=green)
LIGNE
resTransformation=Line from Figure(type=line, x1=10, y1=10, x2=100, y2=10, color=black)
TYPE DE FIGURE INCONNU

resTransformation=Unknown from Figure(type=ellipse, x1=10, y1=10, x2=100, y2=30, color=orange)

Phase3 :

1. lire les **indications** dans les méthodes figureTo...() de **tp.util.FigureUtil**
2. **coder** les **...Record** nécessaires au sein de **tp.j15_16_17.Dto**
3. **coder les méthodes figureTo...() de tp.util.FigureUtil** en utilisant le mot clef var si besoin de variables locales
4. relancer **tp.j15_16_17.SyntheseTestApp.main()**

Résultat attendu :

LIGNE

resTransformation=LineRecord[x1=10, y1=10, x2=100, y2=100, lineColor=red]

RECTANGLE

resTransformation=RectangleRecord[x1=5, y1=5, largeur=115, hauteur=115, fillColor=blue]

CERCLE

resTransformation=CircleRecord[xc=60, yc=60, radius=28, fillColor=green]

LIGNE

resTransformation=LineRecord[x1=10, y1=10, x2=100, y2=10, lineColor=black]

TYPE DE FIGURE INCONNU

resTransformation=FigureRecord[type=ellipse, x1=10, y1=10, x2=100, y2=30, color=orange]

Phase4 :

- lire les **indications** dans les méthodes figureTo...() de **tp.j15_16_17.Dto**
- au niveau de chaque record , redéfinir la méthode toString() en s'appuyant sur les **TextBlocs** et de manière à générer des chaines de caractères au format JSON.
- **relancer** **tp.j15_16_17.SyntheseTestApp.main()**

résultat attendu :

LIGNE

```
resTransformation={
  "x1" : 10,
  "y1" : 10,
  "x2" : 100,
  "y2" : 100,
  "lineColor" : "red"
}
```

RECTANGLE

```
resTransformation={
  "x1" : 5,
  "y1" : 5,
  "largeur" : 115,
  "hauteur" : 115,
  "fillColor" : "blue"
}
```

CERCLE

```
resTransformation={
```

```
"xc" : 60,  
"yc" : 60,  
"radius" : 28,  
"fillColor" : "green"  
}
```

LIGNE

```
resTransformation={  
  "x1" : 10,  
  "y1" : 10,  
  "x2" : 100,  
  "y2" : 10,  
  "lineColor" : "black"  
}
```

TYPE DE FIGURE INCONNU

```
resTransformation={  
  "type" : "ellipse",  
  "x1" : 10,  
  "y1" : 10,  
  "x2" : 100,  
  "y2" : 30,  
  "color" : "orange"  
}
```

5.3. Exo synthèse (HttpClient + Record + Json/Jackson , ...)

Compléter le code de la classe `tp.j15_16_17.HttpJsonRecordTestApp` selon les indications suivantes :

Phase 1 :

Via l'api **HttpClient** , appeler en mode **GET** le web service REST dont l'URL est

https://catfact.ninja/fact

Ce WS facile à appeler (sans api_key) retourne une réponse au format **JSON** de type
{ "fact" : "un fait sur les chats" , "length" : 21_ou_autre }

On pourra effectuer l'appel en mode synchrone ou bien en asynchrone via
`client.sendAsync(req, BodyHandlers.ofString()).thenAccept(resp -> ...);`
si mode asynchrone choisie prévoir pause d'attente en fin du `main()`

Phase 2 :

Via une structure de code de ce type :

```
client.sendAsync(req, BodyHandlers.ofString())  
    .thenApply(resp -> {  
        System.out.println(.....);  
        System.out.println(.....);  
        return resp.body();  
    })  
    .thenApply(*****lambda transformant jsonString en Dto.CatFact*****)  
    .thenAccept(*****lambda affichant le record de type Dto.CatFact*****);
```


on pourra:

- transformer la réponse du format "jsonString" vers le format instance de **DTO.CatFact** (record à coder avec propriétés .fact et .length) en s'appuyant sur l'api jackson databind en version >= 2.12.5 gérant bien les "record"
catFact = jacksonObjectMapper.readValue(catFactAsJsonString, CatFact.class);
- afficher finalement les valeurs du DTO/record java construit viatoString()

Exemple de résultats affichés dans la console:

suite synchrone interpreted by main

recuperation reponse asynchrone / interpreted by ForkJoinPool.commonPool-worker-1

reponse status:200

reponse uri:https://catfact.ninja/fact

reponse type:[application/json]

reponse text:{"fact":"Approximately 1/3 of cat owners think their pets are able to read their minds.", "length":78}

catFact as java record:CatFact[fact=Approximately 1/3 of cat owners think their pets are able to read their minds., length=78]

fin synchrone / interpreted by main

6. TP sur Tests,Mock,Profiling,... (java avancé)

NB : la plupart des énoncés de TP sont basés sur le point de départ suivant :

sous projet "exos_java" de tp/tp_java_avance (https://github.com/didier-tp/java_avance)

6.1. Test unitaire avec assertions

Compléter le code de la classe src/test/java/tp.data.TestPersonne

Partie facultative du Tp :

Compléter le code de la classe src/test/java/tp.calculs.TestParametreCalculsFinanciers en s'appuyant sur @ParameterizedTest et @CsvSource

6.2. Suite de tests

Compléter le code de la classe src/test/java/tp.calculs.CalculsSuite de manière à lancer TestCalculsSimple , TestEmptySerie et TestSerie

Lancer cette suite de tests depuis un IDE (ex : eclipse ou intelliJ)

Lancer si possible (si le temps le permet) cette suite de tests via maven ou via un script (.bat ou .sh ou autre).

6.3. Mockito

- Charger le sous projet "**with_mockito**" de **tp/tp_java_avance** (https://github.com/didier-tp/java_avance) au sein d'un IDE java (ex : eclipse ou intelliJ) .
- Analyser et faire tourner l'exemple **tp.service.TestServiceDevise** .
- Au sein du package **tp.emprunt** s'inspirer de `TestServiceEmpruntSansMock` pour coder **`TestServiceEmpruntAvecMock`**

NB : coder idéalement 2 versions/variantes :

- une variante sans annotations de Mockito
- une seconde variante avec quelques annotations de Mockito

6.4. Vérification de la couverture de code

Installer le plugin maven jaCoCo et s'en servir pour mesurer la couverture des tests

6.5. Test de charge avec JMeter (tp facultatif)

6.6. Profiling

Simulation de fuite mémoire, interprétation de métriques et amélioration ciblée.

